Lecture 06: hexadecimal, floating point, and ASCII

Goals

- convert between binary and hexadecimal representations
- convert a non-integral number to IEEE 754 floating point representation
- describe the limitations of IEEE 754
- describe ASCII, its benefits, and its drawbacks

the decimal numbers we're used to are also called "base 10", because each digit can take on one of ten

therefore, binary is also called "base 2"

two other bases you should know about: base 8 (octal) and base 16 (hexadecimal)

octal

base 8

always begins with a 0

so 0732 = 7 * 8^2 + 3 * 8^1 + 2 * 8^0 = 448 + 24 + 2 = 474

hexadecimal is particularly handy because it translates quickly and easily to/from binary

a hex digit can take one of sixteen different values

0 through 9 (decimal) are represented as 0 through 9 (hex)

10 through 15 (decimal) are represented as A through F (hex)

a group of four binary digits can take one of sixteen different values

thus 1010 is A, 1011 is B, 1100 is C, and so on

if I write the number 101, how do you know what base it is?

you don't: it could be binary, decimal, or hex

in math, the base is often shown in subscript following the number

in CS, a value in hex will always be preceded by "0x" or just "x"

how about representing non-integral values?

another name: "floating point" values, because the decimal point can move around

there's a standard set of rules for this, called IEEE 754, for 32-bit floating point numbers (other rules for l

similar to scientific notation (eg, 6.022 x 10^23)

1 sign bit

8 bits for the exponent

23 bits for the mantissa

example: convert -6 5/16 to IEEE 754

convert the integer portion and the fractional portion separately

we start by examining the integer portion (6) and write its unsigned binary representation to the left of the

the fractional portion (5/16) is a bit more involved

first, we look at the denominator and ask "what power of 2 is this?"

16 is 2^4, so the answer is 4

this means we're going to use 4 bits to represent the fractional portion

then we look at the numerator and write its unsigned binary representation using the number of bits we ju

we retain the sign

and add "* 2^0", which doesn't change the value because it just multiplies by 1, but the reasons will becc

```
-110.0101 * 2^0
```

recall the scientific notation requires a single non-zero digit to the left of the decimal point: IEEE 754 has

if we move the decimal point to the left, however, we have to increase the exponent so that the whole ex
value

since we're moving it to the left two places, we increase the exponent by 2

(if we had to move it to the right—if our number was much smaller than 1—then we would subtract from

the result:

```
-110.0101 = -1.100101 * 2^2
```

we are now mostly ready to write our answer

the left-most bit is the *sign bit*, meaning it represents whether the value is positive or negative

since the number we're working with is negative, its sign bit (the left-most bit) is 1

the next 8 bits give the exponent: in our case 2

to encode the exponent, we add 127 to it and represent the result as an 8-bit unsigned integer

127 + 2 is 129, whose representation as an 8-bit unsigned is 10000001

finally, the mantissa

note that we previously moved the decimal point so that there is exactly one non-zero digit to its left

since we're working with binary, there is only one non-zero digit: one, and so there will *always* be a singl
point at this stage

this means we don't have to actually record it in our 32 bits because The System knows it should always

therefore, the remaining 23 bits are just the bits of the mantissa to the right of the decimal point, extende
10010100000000000000000

full result:

```
1 10000001 10010100000000000000000
^ ^^^^^^^^ ^^^^^^^^^^^^^^^^^^^^^^^
| |        mantissa, leading one dropped, extended with zeroes
| exponent + 127 as unsigned binary
sign bit: 0 for zero/positive, 1 for negative
```

IEEE 754 has undeniable limitations

most importantly: the denominator of the fractional component MUST be a power of 2

this means that we cannot represent numbers like 1/3 exactly

instead, we just pick a huge power of 2 for the denominator and get as close as we can for the numerato

for the majority of cases, this is sufficient precision

if you encounter a situation where you need to be more precise, you would use a different encoding

it is also the case that, as the number we represent with IEEE 754 gets larger, our ability to represent a p reduced

this is because the same bits are used to represent both (the 23-bit mantissa portion)

any thoughts on how to use binary to represent text?

there doesn't seem to be any obvious scheme like that of integers

and while there's some order involved (eg, letters and digits) a lot of characters on the keyboard are uno

one method would be to just arbitrarily assign letters to bit sequences

since most memory is byte-addressable, maybe one letter per byte

and that's exactly what they did

ASCII: American Standard Code for Information Interchange

(run `man ascii` on a Linux or OS X system to see the table)

this is effective

but it's shamefully Anglo-centric

doesn't even accommodate other Western languages like French and German

let alone Arabic, Chinese, Japanese, etc

enter Unicode

a way to encode (describe) characters using variable-length sequences of bytes

meaning that some characters take 1 byte, some take 2, some take 3, and some take 4 (I think 4 is the n

this lets us represent bazillions of characters

there's even a proposal to include Tengwar in the standard

so that covers numbers and letters, which are pretty basic

what about more exciting stuff like audio, video, and images?

there exist standards for these

by "standard" I mean a commonly accepted, published rules to encode, eg, image data as binary and int

JPEG and PNG are just such standards

one way to encode images is to break up an image into a grid of tiny squares (pixels—picture elements)

binary data describing an image could look like the following

- 32-bit unsigned integer indicating width in pixels
- 32-bit unsigned integer indicating height in pixels

- 32-bit quantity indicating color of pixel in top-left corner
- 32-bit quantity indicating color of pixel immediately to the right
- and so on to the end of the row and then continue with the next row

to represent a color using 32 bits, have an 8-bit unsigned integer represent the strength of red, another g another opacity (alpha)

this scheme for representing colors is called RGBA

this is one way to describe images using binary

afaik, no standard uses this exact method

mostly because it's inefficient

JPEG, PNG, etc are much more clever

the point is, however, that the basic types we've seen are often used as building blocks to describe more

mp3 for audio

H.264 for video

then there are ways to describe not just inert data like images and video and audio, but to describe *comm*

TCP/IP

USB (p276 of usb_20.pdf)

again, these use the building blocks

to reiterate

binary data is just a bunch of bits

to get meaning from that data, we need to be told how to interpret it

we've looked at methods to encode lots of different kinds of data as sequences of zeroes and ones

plus a really convenient way to communicate arbitrary sequences of zeroes and ones (hexadecimal)

Mechanical Skills

The following mechanical skills introduced in this lecture are fair-game for future quizzes. You may acces exactly resemble the questions on the quizzes) on `weathertop`.

```
t1p1m05    Convert between binary and hex
t1p1m06    Convert between hex and ASCII
t1p1m07    Convert between binary and IEEE 754
```

Last modified: 09/27/2026 19:17:49